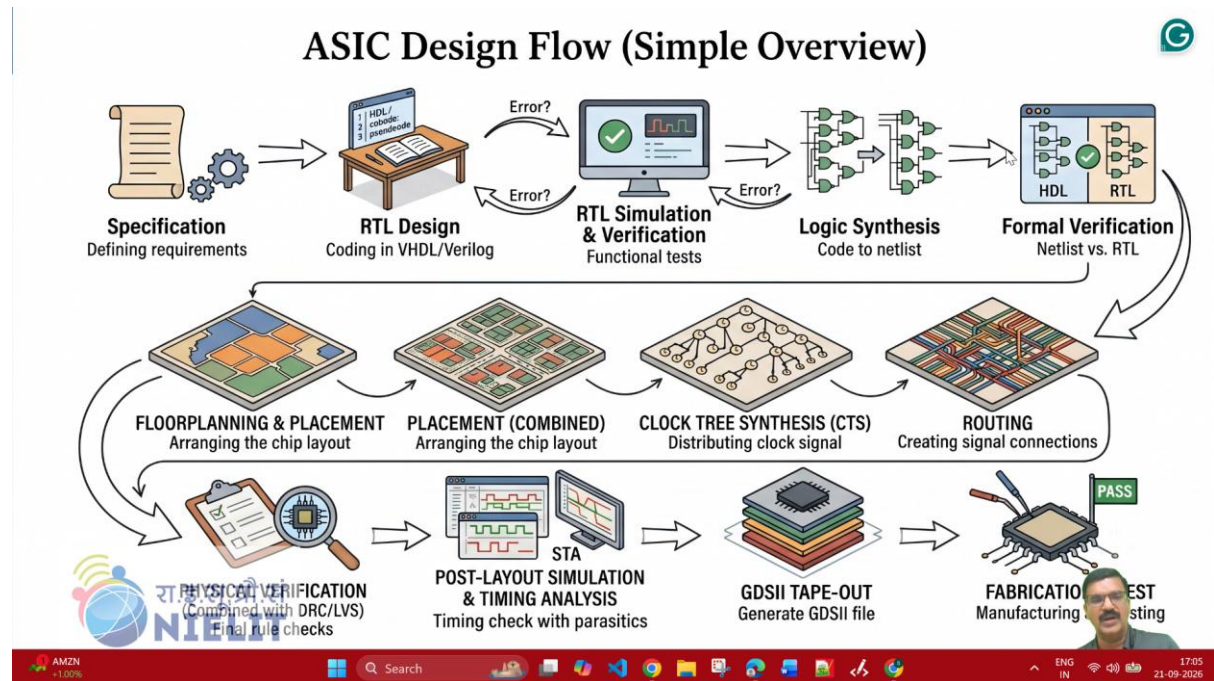


NIELIT : Internship Revision of the VLSI Design Brush the Basics

It's like ASIC class from VIT taught by Jaganath Naidu.



What is RTL — Register Transfer Level?

- Abstraction level between algorithm and gate
- Describes how DATA moves between REGISTERS on each clock edge
- Each line of RTL maps to real hardware:
 - assign = combinational logic
 - always @(posedge clk) = flip-flop
 - if/else = multiplexer
 - case = decoder
- RTL is SYNTHESIZABLE — tools can convert to gates
- Unlike behavioral code which only simulates

```
// 4-bit up counter — RTL example
module counter(
    input clk, rst,
    output reg [3:0] count
);
always @(posedge clk) begin
    if (rst)
        count <= 4'd0;
    else
        count <= count + 1;
end
endmodule
```

Writing Synthesizable RTL — Do's & Don'ts



✓ DO

- Use non-blocking ($\leq$) in always @(posedge clk)
- Use blocking ($=$) in combinational always blocks
- Fully specify sensitivity lists or use always @(*)
- Reset all flip-flops to known state
- Keep FSMs in separate always blocks



✗ DON'T

- Use initial blocks (not synthesizable)
- Infer latches — always cover all branches
- Use delays (#10) in RTL — only for simulation
- Mix blocking and non-blocking in same block
- Leave signals uninitialized in reset



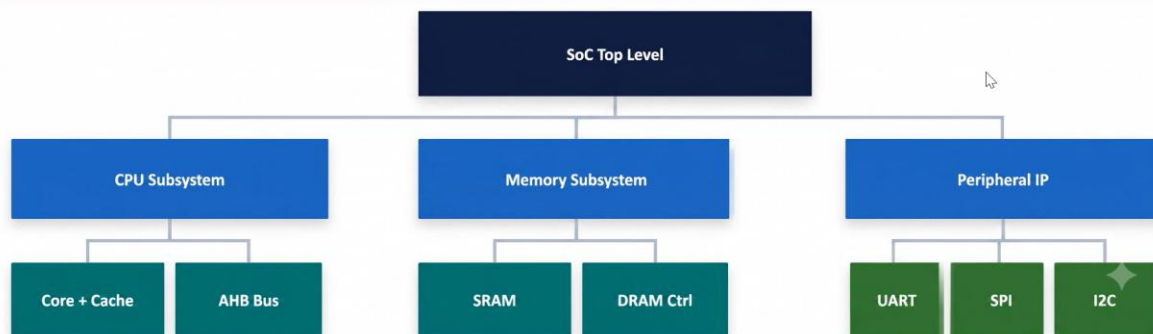
28°C
Light rain

Search



ENG IN 17:23
21-09-2026

Design Hierarchy — Modules, IPs and SoC Integration



- Hard IPs: Pre-designed, verified, process-specific — PLL, IO cells, memory compilers (from foundry PDK)
- Soft IPs: Synthesizable RTL — CPU cores (ARM Cortex-M), AHB interconnect
- Firm IPs: Partially placed/optimized — between hard and soft



28°C
Light rain

Search



ENG IN 17:27
21-09-2026

Why Verification? The Cost of Bugs

2–3×

Verification effort vs design

\$1M+

Average re-spin cost

70%

Bugs found post-silicon

18mo

Extra time for re-tape-out

The Verification Gap

- Intel Pentium FDIV bug (1994): \$475M recall — a hardware divide error that slipped verification
- Mars Climate Orbiter (1999): Wrong unit conversion in software — \$328M satellite lost

'Shift Left' philosophy: Find bugs at RTL simulation — 100× cheaper than post-silicon debug



28°C
Light rain

Search

ENG IN 17:29
21-09-2026

Testbench Architecture



- Stimulus: Random + directed test vectors applied to DUT inputs
- DUT: Your RTL module under test (not modified)
- Reference Model: Golden behavioral model — software or algorithmic
- Scoreboard: Compares DUT output vs reference — flags mismatches
- Coverage: Measures which scenarios have been tested (functional + code coverage)



INFI
-2.26%

Search

ENG IN 17:31
21-09-2026

Formal Verification — Proving Correctness Mathematically

Formal vs Simulation

Simulation: Tests specific stimulus vectors — you only check what you think of.

Formal: Exhaustively proves properties are TRUE for ALL possible inputs — mathematically complete.

Property Checking (Model Checking)

Assert/Assume/Cover SVA properties.
Tool: JasperGold, VC Formal



Equivalence Checking

Prove synthesized netlist matches RTL.
Tool: Conformal, Formality

Clock Domain Crossing (CDC)

Detect metastability risks at async
clock boundaries



AMZN
+1.00%



Search



ENG
IN



17:34
21-09-2020

What is Logic Synthesis?

- Process of converting RTL (HDL) → Gate-level Netlist
- Uses a standard cell library (technology-specific)
- Optimizes for: Timing / Area / Power — pick any two!
- Inputs to synthesis:
 - RTL source files (.v / .sv)
 - Constraints file (.sdc — Synopsis Design Constraints)
 - Technology library (.lib — Liberty format)
 - Physical library (.lef — cell dimensions)
- Output: Gate-level netlist (.v) + timing report + area report



```
assign sum = a ^ b ^ cin;
```

↓ Synthesis

```
XOR2 U1 (.A(a),.B(b),.Z(t1));  
XOR2 U2 (.A(t1),.B(cin),.Z(sum));
```

Gate-level netlist — maps to physical cells



20°C
Light rain



Search



ENG
IN



17:36
21-09-2020

Inside Synthesis — Stages of the Tool Flow

1. Elaboration

Parse RTL, build design hierarchy, check for errors

2. Generic mapping

Convert RTL ops to technology-independent Boolean network (GTECH)

3. Technology mapping

Map GTECH to standard cells from .lib — multiple solutions tried

4. Optimization

Iterative gate-level optimization: timing → area → power trade-offs

5. Netlist outputs

Write .v gate netlist, SDF timing file, area/power reports



28°C
Light rain

Search

ENG IN 19:32
21-09-2020

Physical Design Flow — From Netlist to Layout

1. Design Import

Read synthesized netlist + LEF + SDC

2. Floorplanning

Die area, power rings, macro placement, IO pin assignment

3. Power Planning

VDD/VSS rings, straps — IR drop minimization

4. Placement

Std cell placement → global → legalization → detailed

5. CTS

Clock Tree Synthesis — balance clock skew < 50ps goal

6. Routing

Global → Detailed → Via optimization → DRC fixing



28°C
Light rain

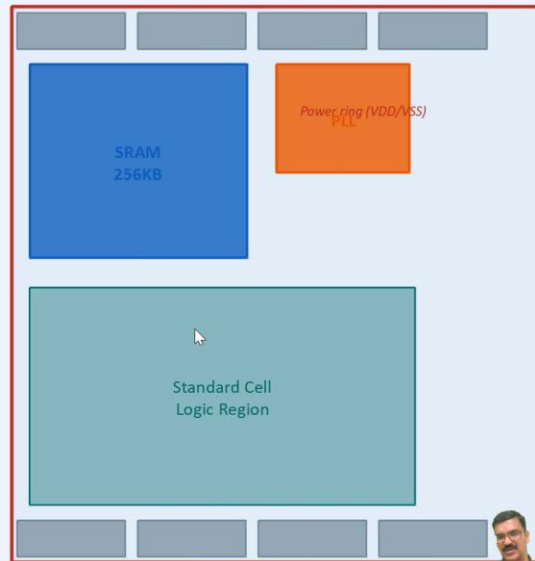
Search

ENG IN 19:34
21-09-2020

Floorplanning — Die Size & Block Placement



- Die size determined by: area utilization (70–80%)
- Power ring: VDD and VSS rings around chip periphery
- Macro placement: Large blocks (SRAM, PLL) placed first
- IO ring: Pad cells placed on chip boundary
- Aspect ratio: Usually $\sim 1:1$ for thermal uniformity
- Halo/blockage: Keep-out zone around macros
- Goal: Minimize wirelength, avoid congestion hotspots



Standard Cell Placement



Global placement

Rough placement minimizing wirelength HPWL. Cells may overlap.

Legalization

Cells snapped to rows, overlaps resolved. No cell outside placement boundary.

Detailed placement

Fine-tune: local swaps, mirroring, padding to improve timing.

Key Placement Metrics & Fixes

- HPWL (Half-Perimeter Wire Length): Primary optimization metric
- Congestion map: Hotspot detection — add blockages or spread cells
- Timing-driven placement: Critical path cells placed close together



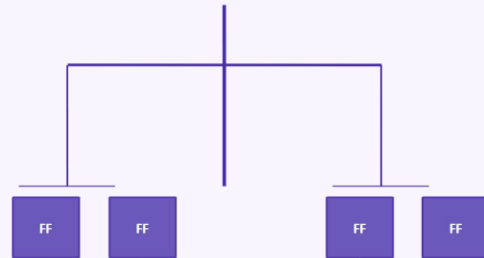
Clock Tree Synthesis (CTS)

Why CTS?

- Clock must reach ALL flip-flops at SAME time
- Clock skew = difference in clock arrival time
- High skew → Hold violations
- CTS inserts buffers/inverters to balance paths
- Target skew < 50–100 ps for 200MHz+ designs
- NDR (Non-Default Rules): wider metal for clock nets
- Shielding: GND/VDD wires beside clock to reduce coupling noise



H-Tree Clock Distribution



Equal path length → equal delay

$$\text{Skew} = |t_{\text{FF1}} - t_{\text{FF4}}| < 50\text{ps}$$



Routing — Connecting the Dots

1. Global routing

Assigns signal to routing regions — guides but does not draw actual wires

2. Track assignment

Assigns each net to specific routing tracks — resolves conflicts

3. Detailed routing

Actual metal + via geometry drawn — DRC-clean connections

4. Via optimization

Replace single vias with double/stacked vias for reliability



Metal stackup: M1 (power), M2-M3 (std cell routing) → M4-M6 (block routing) → M7-M9 (power/clock trunk) — varies by process node





IR Drop

- Resistance of metal wires causes voltage drop: $V = I \times R$
- High IR drop → gates run at lower Vdd → timing failure
- Static IR: Average current analysis
- Dynamic IR: Switching current peaks
- Fix: Add more power straps, widen metal



Electromigration (EM)

- High current density moves metal atoms → wire opens over time
- Reliability concern — not a simulation bug
- Occurs at: vias, narrow wires, bends
- Fix: Widen wires, add parallel paths, double vias
- Verified using IR/EM signoff tools (Voltus, RedHawk)



The DFT Problem: How do you test a chip with millions of gates once it's manufactured?

Scan insertion

Flip-flops connected in a chain → directly readable/writable from test pins. Enables controlled state access.

BIST (Built-in Self Test)

On-chip logic generates test patterns and evaluates results — used for memories (MBIST).

JTAG / Boundary Scan

IEEE 1149.1 standard tests board-level interconnects via 4-pin TAP interface.

ATPG

Automatic Test Pattern Generation — algorithms that generate vectors to detect stuck-at faults.



Signoff Verification — Gates Before Tape-Out



STA (Static Timing Analysis)

Check all timing paths meet setup/hold after extraction. Tool: PrimeTime, Tempus. Zero simulation needed.

DRC (Design Rule Check)

Verify layout obeys foundry geometrical rules — minimum width, spacing, enclosure. Tool: Calibre DRC, Pegasus.

LVS (Layout vs Schematic)

Compare extracted netlist from layout against gate netlist. Catches missing connections, shorts. Tool: Calibre LVS.

ERC (Electrical Rule Check)

Checks floating gates, antenna violations, well-taps. Prevents reliability failures in silicon.

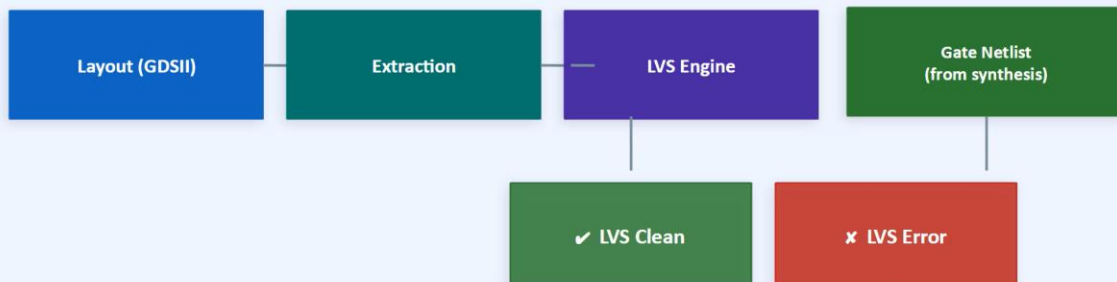
IR/EM Signoff

Full-chip power grid analysis for IR drop and electromigration limits. Tool: Voltus, Redhawk.

Antenna Check

Long metal during etch process accumulates charge → damages gate oxide. Fixed by diodes or antenna jumpers.

LVS — Layout vs Schematic



Common LVS Errors

- Short: Two nets connected in layout but not in schematic
- Open: Net exists in schematic but broken in layout
- Device mismatch: Wrong transistor size or connection — often from missing body tie or substrate contact

ECO Flow & Pre-Tape-Out Checklist



ECO — Engineering Change Order

- Last-minute fix without full re-run
- Types: Functional ECO / Timing ECO / Power ECO
- Spare cells placed during design for ECO use
- Metal-only ECO: Change only metal layers — cheaper re-spin
- Tool-based ECO: Conformal ECO, IC Compiler ECO



Pre-Tape-Out Checklist

- DRC clean — zero violations
- LVS clean — all nets match
- STA signoff — all slack ≥ 0
- IR drop $< 5\%$ Vdd
- EM violations zero
- Functional simulation pass
- DFT test coverage $> 98\%$
- Density fill complete
- GDSII stream-out verified



24°C
Light rain

Search



ENG IN 19:49
21-09-2026

Process Design Kit (PDK) & Foundry Interface



TSMC

3nm, 5nm, 7nm, 16nm

Samsung

3nm, 4nm, 5nm, 8nm

Intel Foundry

Intel 18A, 3nm

GlobalFoundries

12nm, 22nm, 28nm

SCL India

180nm, 0.8 μ m

CY Semi / IISc

Research nodes

What's in a PDK?

- Spice models: Transistor electrical models (TT, FF, SS, FS, SF corners)
- Standard cell library: .lib (timing), .lef (physical), .gds (layout), .v (behavior)
- DRC/LVS runsets: Foundry rules for Calibre/Pegasus verification — NDA protected



24°C
Light rain

Search



ENG IN 19:50
21-09-2026



OpenLane Flow

- Open-source RTL-to-GDSII flow by Efabless
- Uses: Yosys (synthesis) → OpenROAD (P&R) → Magic (DRC/LVS)
- Skywater 130nm open PDK — first openly published PDK
- Free to run — no commercial license needed
- Used in Google-sponsored MPW shuttles
- GitHub: github.com/The-OpenROAD-Project



Commercial vs Open Source

Stage	Commercial	Open Source
Synthesis	DC Compiler	Yosys
P&R	Innovus/ICC2	OpenROAD
STA	PrimeTime	OpenSTA
DRC/LVS	Calibre	Magic/Klayout
Sim	VCS/Xcelium	Verilator/Icarus



Summary — The RTL to GDSII Journey



01 RTL Design

Write clean, synthesizable HDL. Avoid latches, use resets.

02 Simulation

Verify every corner. Coverage = confidence. Shift left.

03 Synthesis

Constraints drive optimization. Wrong SDC = failed silicon.



04 Physical Design

Floor → Power → Place → CTS → Route. Iterate on timing.

05 Signoff

DRC + LVS + STA + IR/EM — all clean before release.

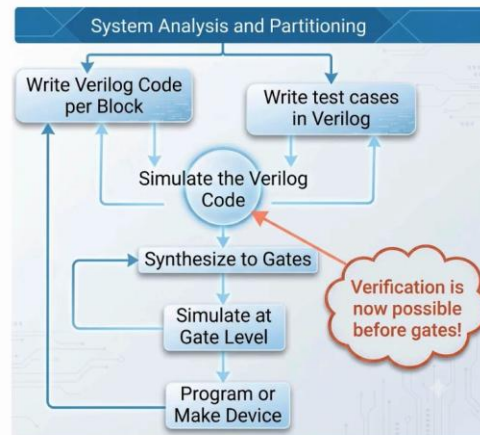
06 Tape-Out

GDSII + complete package to foundry. Irreversible!



Verilog HDL : Language

Motivation: Look Ahead Synthesis



- The functional simulation can be completed, before the synthesis
- It saves a lot of time and effort in synthesis since the functionality of the design is validated.



Data Types: wire vs. reg

Primary Data Types

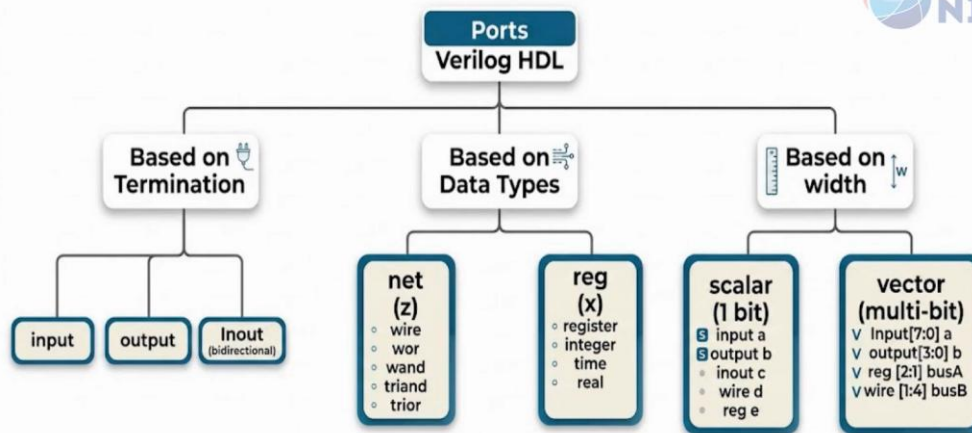
- **wire**: Physical connection — models combinational logic; driven by **assign** or module outputs; **cannot store value**
- **reg**: Stores a value between assignments — used in **always** blocks; does **NOT** always map to a flip-flop
- **integer**: 32-bit signed — used in loops and testbenches; NOT synthesizable as hardware
- **real**: Floating-point — simulation only, never synthesizable
- **parameter**: Compile-time constant — for bus widths, delays, state encoding
- **localparam**: Like parameter but cannot be overridden at instantiation — safer for internal constants

Property	wire	reg
Driven by	assign / module output	always / initial block
Stores value?	No (combinational)	Yes (holds last value)
Default value	Z (high-impedance)	X (unknown)
Synthesizes to	Wire / net	FF or combinational logic
Use case	Connections, buses	Sequential & combo logic

⚠ A common beginner mistake: using **wire** as the LHS of an **always** block, or using **reg** with **assign**. The compiler will error out with an error message that is not type to context.



Port Types



What are Verilog Operators?

Verilog operators are symbols used to perform operations on one or more operands. They are essential for modeling digital logic, from simple gates to complex architectural behaviors. Operators in Verilog are categorized based on their function and the number of operands they require.

Functional Categories

Arithmetic, Logical, Relational, Equality, Bitwise, Shift, and Reduction.

Operand Counts

Unary (1 operand), Binary (2 operands), and Ternary (3 operands).

Verilog Operators

Arithmetic

`+` `-` `*` `/` `%` `**`

Synthesizable except division/power in most tools

Bitwise

`&` `|` `^` `~` `^~`

AND, OR, XOR, NOT, XNOR — operate bit-by-bit on vectors

Logical

`&&` `||` `!`

Return 1-bit result; treat any non-zero value as true

Relational

`==` `!=` `<` `>` `<=` `>=`

`===` and `!==` also compare X/Z values

Shift

`<<` `>>` `<<<` `>>>`

Logical and arithmetic shifts; `>>>` preserves sign bit

Reduction

`&a` `|a` `^a`

Reduces all bits of vector to single bit AND/OR/XOR

Concatenation

`{a, b, c}` joins signals

`{4{a}}` replicates a 4 times

Conditional (Ternary)

`cond ? true : false`

Synthesizes directly to a multiplexer



Operators in Verilog

×

 Arithmetic Operators	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
 Relational Operators	<code><</code> <code><=</code> <code>></code> <code>>=</code>
 Equality Operators	<code>==</code> <code>!=</code> <code>===</code> <code>!==</code>
 Logical Operators	<code>!</code> <code>&&</code> <code> </code>
 Bitwise and Unary Reduction Operators	<code>~</code> <code>&</code> <code> </code> <code>^</code> <code>^~</code>
 Shift Operators	<code>>></code> <code><<</code>
 Conditional Operators	Concatenations <code>{ }</code>
 Bitwise and Unary Reduction Operators <code>? :</code>	Replication <code>{ { } }</code>



Relational Operators

Relational operators compare two operands and return a logical value

Operator	Name	Comments
>	Greater than	Returns 1 if true
<	Less than	Returns 1 if true
>=	Greater than or equal to	Returns 1 if true
<=	Less than or equal to	Returns 1 if true



Equality Operators

Used for bit-by-bit comparisons. Reference:

Operator	Name	Comments
==	Equality	result unknown if x or z
!=	Inequality	result unknown if x or z
===	Case Equality	equal including x or z
!==	Case Inequality	unequal including x or z



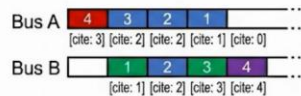
Vectors & Arrays

Vector represents bus

- Left number is MSB
- Vector assignment by position

```
// vectors
wire [3:0] busA; // busA[3] is MSB
reg [1:4] busB; // busB[1] is MSB
reg [1:0] busC;

// wire [3:0] busA; // busA[3] is MSB
// subfield access and positional assignment
busC = busA[2:1]; // busC[1] = busA[2];
// busC[0] = busA[1];
```



Array is a collection of the same data type

```
// array
integer my_int[1:5]; // 5 integer variables
reg my_reg[0:99];
reg [7:0] my_mem[0:9];
reg my_x[7:0][0:10];
```



my_x: two-dimensional one-bit register.

my_mem: 8-bit memory from address 0 to 9.

Array Assignments : Examples

Array declarations

```
reg [7:0] mema[0:255]; // declares a memory mema of 256 8-bit registers. The indices are 0 to 255
reg arrayb[7:0][0:255]; // declare a two-dimensional array of one bit registers
wire w_array[7:0][5:0]; // declare array of wires
integer inta[1:64]; // an array of 64 integer values
time chng_hist[1:1000] // an array of 1000 time values
integer t_index;
```

Assignment to array elements

```
mema = 0; // Illegal syntax- Attempt to write to entire array
arrayb[1] = 0; // Illegal Syntax - Attempt to write to elements [1][0]..[1][255]
arrayb[1][12:31] = 0; // Illegal Syntax - Attempt to write to elements [1][12]..[1][31]
mema[1] = 0; // Assigns 0 to the second element of mema
arrayb[1][0] = 0; // Assigns 0 to the bit referenced by indices [1][0]
inta[4] = 33559; // Assign decimal number to integer in array
chng_hist[t_index] = $time; // Assign current simulation time to element addressed by integer index
```


- Memories can be modeled as **array of registers**
- Each element in the array represents a **'word'**
- Each 'word' can be one bit or multiple bits wide.

```
reg mem1bit[0:1023];           //1k 1-bit words
reg [7:0] membyte[0:1023];     //1k 8-bit words

membyte[511] //fetches 1 byte word at 511
```



DAY 2 : Hierarchical Design

Hierarchical Design & Module Instantiation

Design Approaches

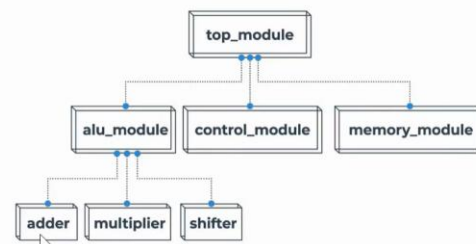
- **Top-down:** Define system architecture first, then implement sub-modules — promotes parallel team development
- **Bottom-up:** Build and verify leaf modules first, then integrate — common in IP reuse scenarios

Instantiation Syntax

```
// Named port connection (recommended)
adder u1 (
  .a (in_a),
  .b (in_b),
  .sum (result)
);

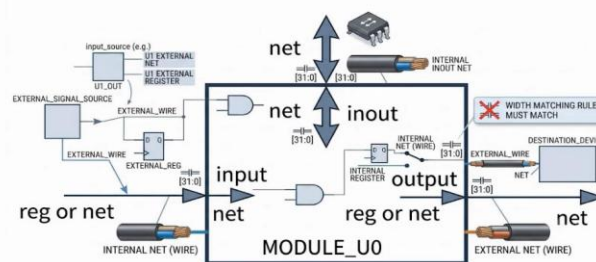
// Parameter override - 16-bit instance
adder #(.WIDTH(16)) u2 (
  .a(x), .b(y), .sum(z)
);
```

Module Hierarchy Example



✓ **Benefits:** Modular verification, IP reuse, clear design partitioning, easier debugging — hierarchy is the foundation of scalable design.

Port Connection Rules



- **Input ports** should be of **net** data type (wire) and can be connected to register to the external world. net as well as
- **Output ports** may be register or a net internally but should be connected to net to the external world.
- **Width Matching :** It is illegal to connect an internal and External items of different sizes making Inter-module port connections.

